Experiment - 7

Motion Estimation using Optical Flow

Lucas-Kanade Sparse Optical Flow

Farneback Dense Optical Flow

In [5]:

```
import cv2 import numpy as
np import imageio.v2 as
imageio
from IPython.display import Image, display cap

= cv2.VideoCapture("videoo.mp4")

if not cap.isOpened():
    print("Video not found!")
else:
    ret, old = cap.read()     old = cv2.resize(old,
None, fx=0.4, fy=0.4)     old_gray =
cv2.cvtColor(old, cv2.COLOR_BGR2GRAY)

    p0 = cv2.goodFeaturesToTrack(old_gray, 50, 0.3, 7)
mask = np.zeros_like(old)     result = []

    lk = dict(         winSize=(15, 15),
maxLevel=1,         criteria=(cv2.TERM_CRITERIA_EPS |
cv2.TERM_CRITERIA_COUNT, 10, 0.03)
    )

count = 0

    while len(result) < 60:
        ret, frame = cap.read()
if not ret:             break
count += 1

        if count % 3 != 0:
continue

        frame = cv2.resize(frame, None, fx=0.4, fy=0.4)
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Lucas-Kanade
if p0 is not None:
            p1, st, _ = cv2.calcOpticalFlowPyrLK(
old_gray, gray, p0, None, **lk
            )

            if p1 is not None:
new = p1[st == 1]
old_pts = p0[st == 1]
```

```
                for n, o in zip(new, old_pts):
                    x1, y1 = map(int, n)
x2, y2 = map(int, o)                        mask =
cv2.line(                           mask, (x1, y1),
(x2, y2),
                        (255, 0, 0), 2
                    )
                    frame = cv2.circle(
frame, (x1, y1), 3,
                        (0, 255, 0), -1
                    )

                if len(new) > 0:
p0 = new.reshape(-1, 1, 2)

        # Farneback
        flow = cv2.calcOpticalFlowFarneback(
old_gray, gray, None,
            0.5, 2, 9, 2, 3, 1.1, 0
)

        mag, ang = cv2.cartToPolar(
flow[..., 0], flow[..., 1]
        )

        hsv = np.zeros_like(frame)         hsv[..., 1] = 255
hsv[..., 0] = (ang * 180 / np.pi / 2).astype(np.uint8)
hsv[..., 2] = cv2.normalize(
            mag, None, 0, 255, cv2.NORM_MINMAX
        ).astype(np.uint8)

        dense = cv2.cvtColor(
hsv, cv2.COLOR_HSV2BGR
        )         vectors =

frame.copy()

        for y in range(0, frame.shape[0], 20):
for x in range(0, frame.shape[1], 20):
                fx, fy = flow[y, x]
cv2.arrowedLine(
vectors, (x, y),
                    (int(x + fx), int(y + fy)),
                    (0, 255, 255), 1
                )         lk_result =

cv2.add(frame, mask)

        cv2.putText(lk_result, "Lucas-Kanade",
(10, 25), 0, 0.6, (0,255,255), 2)
cv2.putText(dense, "Farneback Dense",
                    (10, 25), 0, 0.6, (0,255,255), 2)
cv2.putText(vectors, "Motion Vectors",
(10, 25), 0, 0.6, (0,255,255), 2)

        combined = np.hstack(
            (lk_result, dense, vectors)
)
```

```
        result.append(
cv2.cvtColor(
combined,
                cv2.COLOR_BGR2RGB
            )         )

old_gray = gray.copy()

cap.release()

    imageio.mimsave(
        "Optical_Flow_Result.gif",
result,          duration=0.08,
loop=0
    )     print("Experiment completed

successfully!")

    display(
Image(
            filename="Optical_Flow_Result.gif",
width=1000
        )
    )
```

```
Experiment completed successfully!
<IPython.core.display.Image object>
```

